

a) ¿Cuál es la limitación de Singleton frente a DIP, y cómo se resuelve?

- **Limitación:** El patrón Singleton viola el **Principio de Inversión de Dependencias (DIP)** porque las clases que lo utilizan dependen de una **implementación concreta** (la clase Singleton) en lugar de una abstracción (interfaz). Esto crea un acoplamiento fuerte, dificulta el testing unitario (no puedes mockear fácilmente un Singleton) y hace que el código sea rígido.
- **Solución:** Se resuelve **inyectando la dependencia** a través de una interfaz. En lugar de que la clase llame directamente a `MiSingleton.getInstance()`, se define una interfaz (ej. `IConfiguracion`) y se pasa una instancia de ella al constructor de la clase que la necesita. El contenedor de inyección de dependencias (como Spring) se encarga de instanciar el Singleton y pasarlo.

b) ¿En cuántos lugares del proyecto debe aparecer getInstance()? ¿Cuáles?

Debe aparecer en **muy pocos lugares**, idealmente en la **capa de configuración o arranque** (main, fábrica, o contenedor de IoC).

- **¿Cuáles?** Principalmente en el punto de entrada de la aplicación (main) o en una clase de tipo Factory que se encargue de construir el grafo de objetos. **Nunca** debe aparecer dentro de la lógica de negocio (ej. dentro de un método de Compra o Cliente), ya que eso acoplaría la lógica al Singleton.

c) Singleton y Builder usan los dos un constructor privado. ¿Para qué sirve en cada uno?

- **Singleton:** Sirve para **impedir que se creen múltiples instancias** de la clase desde fuera. Obliga a que todo el mundo use el método `getInstance()` para obtener la única instancia existente.
- **Builder:** Sirve para **obligar a que el objeto se construya a través del Builder**. Evita que se salten las validaciones o que se creen objetos en un estado inconsistente. El constructor privado solo es accesible desde la clase anidada Builder.

d) De estas clases de tu proyecto, ¿cuál llevaría Builder y cuál no? Justifiquen en una línea:

- **Cliente, Funcion, Combo: No** llevarían Builder. *Justificación:* Son clases simples con pocos atributos y no tienen validaciones complejas ni parámetros opcionales; un constructor normal es suficiente.
- **Compra: Sí** llevaría Builder. *Justificación:* Tiene múltiples atributos, algunos son opcionales (combo, puntos) y requiere validaciones estrictas antes de crearse.

e) ConfiguracionCine es Singleton y Builder de Compra necesita el IVA que ella guarda. ¿El Builder debe llamar a ConfiguracionCine.getInstance() por dentro? ¿Por qué?

- **No, no debería.**
- **¿Por qué?** Porque eso crearía un **acoplamiento oculto** (dependencia oculta). El Builder dependería directamente de una implementación concreta (el Singleton), violando el DIP. Lo correcto es que el IVA se pase como **parámetro** al método build() o al constructor del Builder, o que se inyecte la interfaz de configuración en el Builder. Así el Builder no sabe de dónde viene el IVA, solo lo usa.